

100

Pendekanti Srihas Gupta

Other

PDF generated at: 8 Jun 2026 04:17:13 UTC

[View this report on HackerRank](#)

Score

100% • 80.0 / 80
scored in CodePath TIP103: Unit 1 Assessment,
Version A - Summer 2026 in 51 min 11 sec on 7
Jun 2026 20:24:09 PDT

Integrity Issues

Medium

Integrity Summary

Detected code similarity with other submissions

Candidate Information

Email	srihasgupta@gmail.com
Test	CodePath TIP103: Unit 1 Assessment, Version A - Summer 2026
Candidate Packet	View
Taken on	7 Jun 2026 20:24:09 PDT
Time taken	51 min 11 sec/ 90 min
Personal Member ID	154368
Email Address with CodePath	srihasgupta@gmail.com
Github username with CodePath	srihas115
Invited by	CodePath

Skill Distribution



There is no associated skills data that can be shown for this assessment

Tags Distribution



There is no associated tags data that can be shown for this assessment

Questions

Coding Questions • 60 / 60

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Matrix Traversal Coding	-	20/20	-	6 min 56 sec	
2	Flowerbed Coding	-	20/20	-	18 min 12 sec	

3	Merge Sorted List Coding	-	20/20		-	19 min 23 sec	
---	--------------------------	---	-------	--	---	---------------	--

Multiple Choice • 20 / 20

No.	Question	Skill	Score	Code Quality	Time Taken	Status
4	What is the output of the following code snippet? Multiple Choice	-	5/5	-	20 sec	
5	What is the output of the following code snippet? Multiple Choice	-	5/5	-	1 min 28 sec	
6	Find the bug! Coding	-	5/5	-	49 sec	
7	Find the bug! Coding	-	5/5	-	55 sec	

1. Matrix Traversal

Correct

Coding

Question description

Write a function `get_sum_of_odds()` that accepts a 2D list of integers `matrix`. Return a list in the format `[num_odds, sum]` where `num_odds` represents the number of odd numbers in the matrix, and `sum` represents the sum of all odd numbers in the matrix.

Example Usage #1:

```
matrix = [  
    [1, 2, 3, 4],  
    [5, 6, 7, 8],  
    [9, 10, 11, 12]  
]
```

Expected Output: [6, 36]

Example Usage 2:

```
matrix2 = [  
    [10, -2],  
    [-3, 5],  
    [4, 8]  
]
```

Expected Output: [2, 2]

Example Usage #3:

```
matrix = []
```

Expected Output: [0, 0]

Candidate's Solution

Language used: Python 3

```
1 #!/bin/python3  
2  
3 import math  
4 import os  
5 import random  
6 import re  
7 import sys  
8 import ast
```

```
9
10 #
11 # Complete the 'get_sum_of_odds' function below.
12 #
13 # The function is expected to return an INTEGER_ARRAY.
14 #
15
16 def get_sum_of_odds(matrix):
17     # Write your code here
18     num_odds = 0
19     sum_odds = 0
20
21     for row in matrix:
22         for num in row:
23             if (num % 2 == 1):
24                 num_odds += 1
25                 sum_odds += num
26
27     return [num_odds, sum_odds]
28
29 if __name__ == '__main__':
30     outfile = open(os.environ['OUTPUT_PATH'], 'w')
31     input_data = input()
32     while (input_data != "END"):
33         matrix = ast.literal_eval(input_data)
34
35         result = get_sum_of_odds(matrix)
36         outfile.write(str(result) + '\n')
37         input_data = input()
38     outfile.close()
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Regular Matrix with Mixed Integers	Easy	Hidden	Success	0	0.0254 sec	10.9 KB
Matrix with Negative and Positive Integers	Easy	Hidden	Success	0	0.0391 sec	10.9 KB

Empty Matrix	Easy	Hidden	Success	0	0.0275 sec	10.9 KB
Matrix with No Odd Numbers	Easy	Hidden	Success	0	0.0304 sec	10.9 KB
Matrix with All Odd Numbers	Easy	Hidden	Success	0	0.0296 sec	10.9 KB
Matrix with Mixed Positive and Negative Integers	Easy	Hidden	Success	0	0.0328 sec	10.9 KB
Single Element Matrix with Odd Number	Easy	Hidden	Success	0	0.0353 sec	10.9 KB
Single Element Matrix with Even Number	Easy	Hidden	Success	0	0.0259 sec	10.9 KB
Matrix with Zero	Easy	Hidden	Success	0	0.0258 sec	10.9 KB
Pass/Fail Case	Easy	Hidden	Success	20	0.0283 sec	10.9 KB

 No comments.

2. Flowerbed

Coding

 Correct

Question description

You have a long flowerbed in which some of the plots are planted, and some are not. However, flowers cannot be planted in **adjacent** plots.

Given an integer array `flowerbed` containing 0's and 1's, where 0 means empty and 1 means not empty, and an integer `n`, return `True` if `n` new flowers can be planted in the `flowerbed` without violating the no-adjacent-flowers rule and `False` otherwise.

Hint: When deciding where to plant a new flower, focus on each plot in the `flowerbed` and check its neighboring plots. You only need to consider the plot directly before and directly after the current plot to determine if a flower can be planted there. Remember that the flowerbed is linear, so you don't need to worry about wrapping around.

Example 1:

Input: `flowerbed = [1,0,0,0,1]`, `n = 1`

Output: `true`

Example 2:

Input: `flowerbed = [1,0,0,0,1]`, `n = 2`

Output: `false`

Candidate's Solution

Language used: **Python 3**

```
1  #!/bin/python3
2
3  import math
4  import os
5  import random
6  import re
7  import sys
8  import ast
9
10
11 #
12 # Complete the 'can_place_flowers' function below.
13 #
14 # The function is expected to return a BOOLEAN.
15 # The function accepts following parameters:
16 # 1. INTEGER_ARRAY flowerbed
17 # 2. INTEGER n
```

```
18 #
19
20 def can_place_flowers(flowerbed, n):
21     # Write your code here
22     placed = 0
23
24     left_empty = False
25     right_empty = False
26
27     for i in range(len(flowerbed)):
28         if (i == 0 or flowerbed[i-1] == 0):
29             left_empty = True
30         if (i == len(flowerbed) - 1 or flowerbed[i + 1] == 0):
31             right_empty = True
32
33         if (flowerbed[i] == 0 and left_empty and right_empty):
34             flowerbed[i] = 1
35             placed += 1
36
37         left_empty = False
38         right_empty = False
39
40
41     return placed >= n
42
43 if __name__ == '__main__':
44     outfile = open(os.environ['OUTPUT_PATH'], 'w')
45     input_data = sys.stdin.read().strip().splitlines()
46
47     for line in input_data:
48         match = re.match(r"(\[.*\]),\s*(\d+)", line)
49         if match:
50             flowerbed_str = match.group(1)
51             n_str = match.group(2)
52
53             flowerbed = ast.literal_eval(flowerbed_str)
54             n = int(n_str)
55             result = can_place_flowers(flowerbed, n)
56             outfile.write(str(result) + '\n')
57     outfile.close()
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
----------	------------	------	--------	-------	------------	-------------

Enough empty plots to plant the required number of flowers	Easy	Hidden	Success	0	0.0263 sec	11 KB
Not enough empty plots to plant the required number of flowers	Easy	Hidden	Success	0	0.0304 sec	11 KB
Flowerbed is already fully planted	Easy	Hidden	Success	0	0.0259 sec	11 KB
Flowerbed is entirely empty	Easy	Hidden	Success	0	0.0309 sec	11 KB
Edge case with one plot in the flowerbed	Easy	Hidden	Success	0	0.0263 sec	11 KB
Edge case with one plot in the flowerbed	Easy	Hidden	Success	0	0.0352 sec	11 KB
Required number of flowers to plant is zero	Easy	Hidden	Success	0	0.0365 sec	11 KB
Required number of flowers to plant is zero	Easy	Hidden	Success	0	0.0315 sec	11 KB

Flowerbed has alternating empty and planted plots	Easy	Hidden	Success	0	0.0321 sec	11 KB
Flowerbed has alternating empty and planted plots	Easy	Hidden	Success	0	0.0263 sec	11 KB
Flowerbed with large number of plots	Easy	Hidden	Success	0	0.0327 sec	11 KB
Pass/Fail Case	Easy	Hidden	Success	20	0.0288 sec	11 KB

🚫 No comments.

3. Merge Sorted List

✍️ Correct

Coding

Question description

Write a function `merge_sorted_lists()` that accepts two sorted lists `lst1` and `lst2` as parameters and merges them into a single sorted list.

Example:

Input: `lst1 = [1, 3, 5]`, `lst2 = [2, 4, 6]`

Output: `[1, 2, 3, 4, 5, 6]`

Candidate's Solution

Language used: Python 3

```
1  #!/bin/python3
2
3  import math
4  import os
5  import random
6  import re
7  import sys
8  import ast
9
10 #
11 # Complete the 'merge_sorted_lists' function below.
12 #
13 # The function is expected to return an INTEGER_ARRAY.
14 # The function accepts following parameters:
15 # 1. INTEGER_ARRAY lst1
16 # 2. INTEGER_ARRAY lst2
17 #
18
19 def merge_sorted_lists(lst1, lst2):
20     # Write your code here
21     merged_list = []
22     p1 = 0
23     p2 = 0
24
25     while (p1 < len(lst1) and p2 < len(lst2)):
26         if (lst1[p1] > lst2[p2]):
27             merged_list.append(lst2[p2])
28             p2 += 1
29         else:
30             merged_list.append(lst1[p1])
31             p1 += 1
32
33     while (p1 < len(lst1)):
34         merged_list.append(lst1[p1])
35         p1 += 1
36
37     while (p2 < len(lst2)):
38         merged_list.append(lst2[p2])
39         p2 += 1
40
41     return merged_list
42
43 if __name__ == '__main__':
44     outfile = open(os.environ['OUTPUT_PATH'], 'w')
```

```

45 input_data = sys.stdin.read().strip()
46
47 input_lines = input_data.splitlines()
48
49 for line in input_lines:
50     lists = re.split(r'\s*,\s*(?=\s*)', line)
51
52     if len(lists) != 2:
53         raise ValueError("Input format is incorrect. Expected two lists
54 separated by a comma.")
55
56     lst1 = ast.literal_eval(lists[0].strip())
57     lst2 = ast.literal_eval(lists[1].strip())
58
59     # Merge the lists
60     result = merge_sorted_lists(lst1, lst2)
61
62     # Print the result
63     outfile.write(str(result) + '\n')
64     outfile.close()

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Basic Case with Non-Overlapping Elements	Easy	Hidden	Success	0	0.0264 sec	11 KB
Both Lists Empty	Easy	Hidden	Success	0	0.0271 sec	11 KB
One List Empty	Easy	Hidden	Success	0	0.0375 sec	11 KB
One List Empty (Reverse)	Easy	Hidden	Success	0	0.0298 sec	11 KB

Lists with Duplicate Elements	Easy	Hidden	Success	0	0.0271 sec	11 KB
Lists with Duplicate Elements	Easy	Hidden	Success	0	0.027 sec	11 KB
Lists with Different Lengths	Easy	Hidden	Success	0	0.0261 sec	11 KB
Lists with All Elements the Same	Easy	Hidden	Success	0	0.0288 sec	11 KB
Interleaved Elements	Easy	Hidden	Success	0	0.0275 sec	10.9 KB
Lists with Negative Numbers	Easy	Hidden	Success	0	0.0288 sec	11 KB
Pass/Fail Case	Easy	Hidden	Success	20	0.0277 sec	11 KB

No comments.

4. What is the output of the following code snippet?

Correct

Multiple Choice

Question description

```
def mystery_function(word):  
  
    start = 0  
    end = len(word) - 1  
  
    while start < end:  
        if word[start] != word[end]:  
            return False  
        start += 1  
        end -= 1  
    return True  
  
word = "kayak"  
result = mystery_function(word)  
print(result)
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

☒ True



☐ False

☐ Throws an error because strings don't support slicing.

☐ Causes an infinite loop, causing the program to crash

 No comments.

5. What is the output of the following code snippet?

✓ Correct

Multiple Choice

Question description

```
def mystery_function(nums):  
    count = 0  
    max_count = 0  
    for i in range(len(nums)):  
        if nums[i] > 0:  
            count += 1  
        else:  
            if count > max_count:  
                max_count = count  
            count = 0  
    if count > max_count:  
        max_count = count  
    return max_count  
  
result = mystery_function([1, 2, -3, 4, 5, -6, 7, 8, 9])  
print(result)
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

☐ 2

☒ 3



☐ 4



5

 No comments.

6. Find the bug!

 Correct

Coding

Question description

The following function contains a bug. `sum_positives` should accept a list of integers `lst` and return the sum of all positive values in `lst`.

For example, if we passed in `[-1, 2, 3, 4, -5]`, `sum_positives` should return 9.

Which of the following options will fix the bug?

```
def sum_positives(lst):
    total = 0
    for num in lst:
        if num > 0:
            total = num
    return total
```

Candidate's Solution

Language used: Python 3

```
1 #!/bin/python3
2
3 import math
4 import os
5 import random
6 import re
7 import sys
8 import ast
```

```
9
10
11
12 #
13 # Complete the 'sum_positives' function below.
14 #
15 # The function is expected to return an INTEGER.
16 # The function accepts INTEGER_ARRAY lst as parameter.
17 #
18
19 def sum_positives(lst):
20     total = 0
21     for num in lst:
22         if num > 0:
23             total += num
24     return total
25 if __name__ == '__main__':
26     input_str = sys.stdin.read().strip()
27     input_list = ast.literal_eval(input_str)
28     result = sum_positives(input_list)
29     print(result)
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Pass/Fail Case	Easy	Hidden	Success	5	0.0312 sec	10.9 KB

 No comments.

7. Find the bug!

Correct

Coding

Question description

The provided code incorrectly implements the function `sum_matrix`. Given a 2D list of integers `matrix`, `sum_matrix` should return the sum of all elements in `matrix`.

Identify any bug(s) within the given implementation and correct the code so that it successfully passes the provided test cases.

```
def sum_matrix(matrix):  
    row_length = len(matrix[0])  
    for row in matrix:  
        for j in range(row_length):  
            total += row[j]  
    return total
```

Candidate's Solution

Language used: Python 3

```
1  #!/bin/python3  
2  
3  import math  
4  import os  
5  import random  
6  import re  
7  import sys  
8  import ast  
9  
10  
11  
12  #  
13  # Complete the 'sum_matrix' function below.  
14  #  
15  # The function is expected to return an INTEGER.  
16  # The function accepts INTEGER_ARRAY matrix as parameter.  
17  #  
18  
19  def sum_matrix(matrix):  
20      total = 0  
21      for row in matrix:  
22          for j in row:  
23              total += j  
24      return total  
25  if __name__ == '__main__':
```

```
26 input_str = sys.stdin.read().strip()
27 input_list = ast.literal_eval(input_str)
28 result = sum_matrix(input_list)
29 print(result)
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Pass/Fail Case	Easy	Hidden	Success	5	0.0328 sec	10.9 KB

 No comments.